



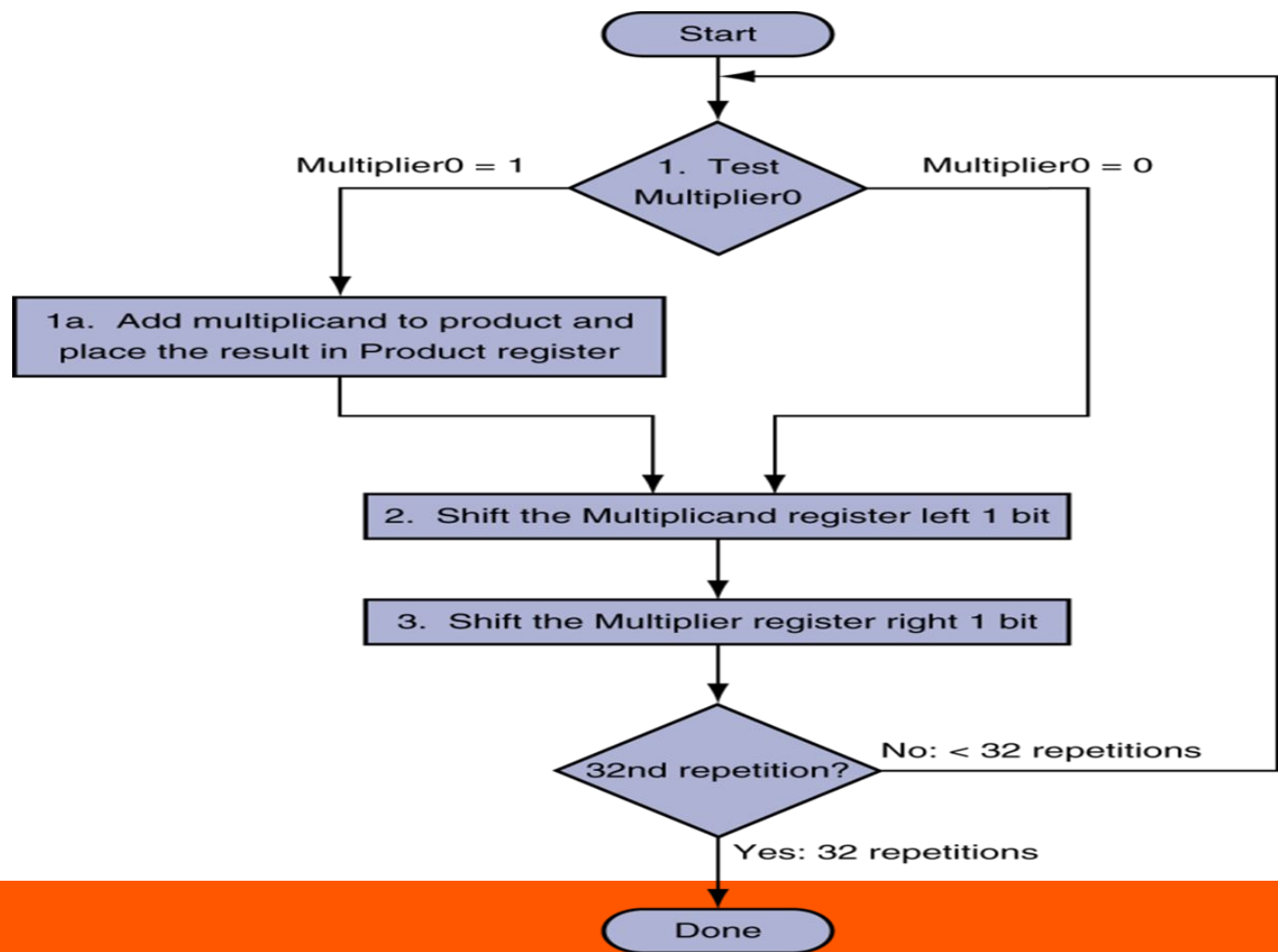
CSE 3313: Computer Architecture

Sidratul Tanzila Tasmi

Lecturer, Department of Computer Science and Engineering
United International University

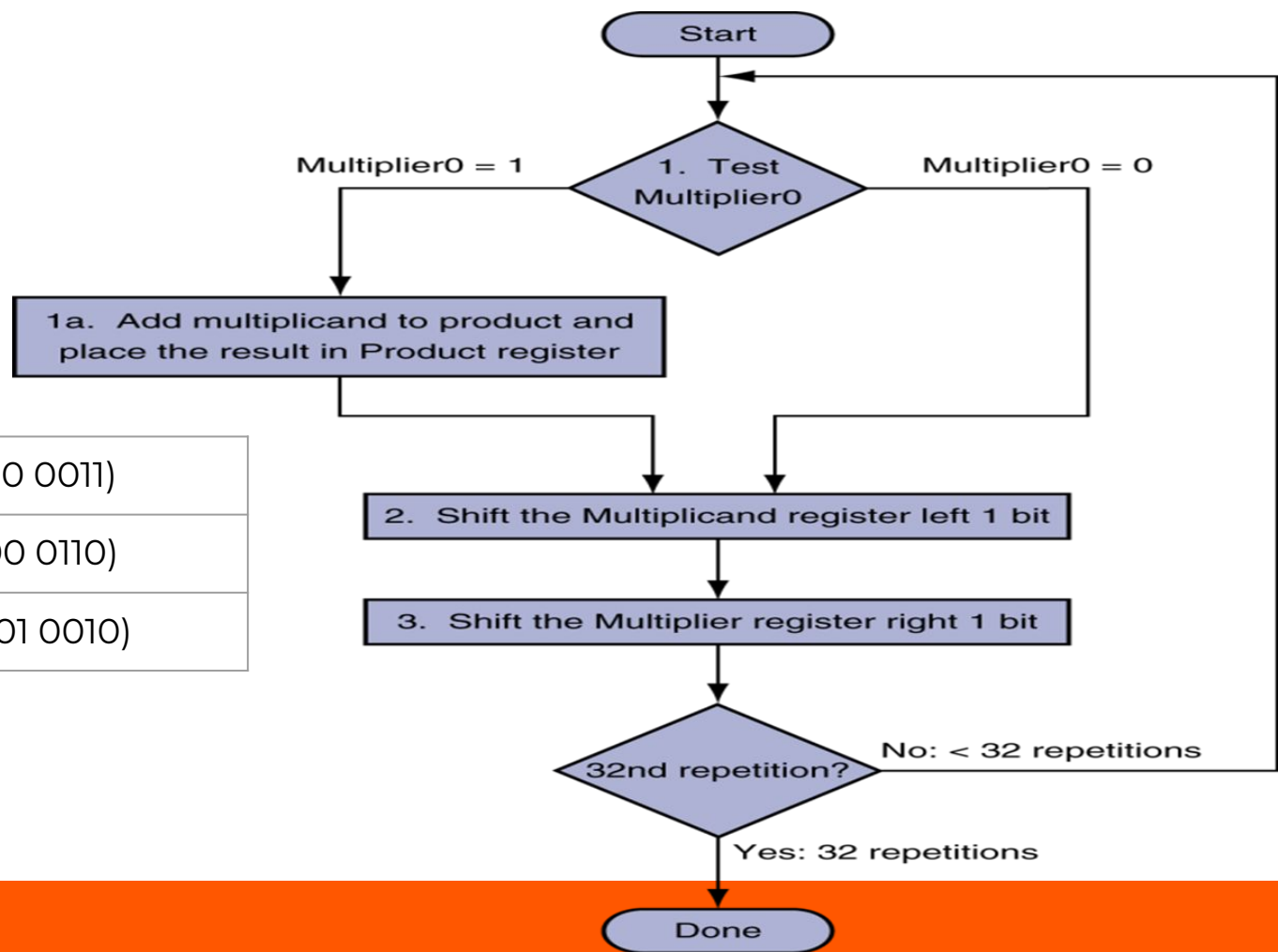
Shift and Add Multiplication Algorithm

It is a process to bitwise multiply two numbers.



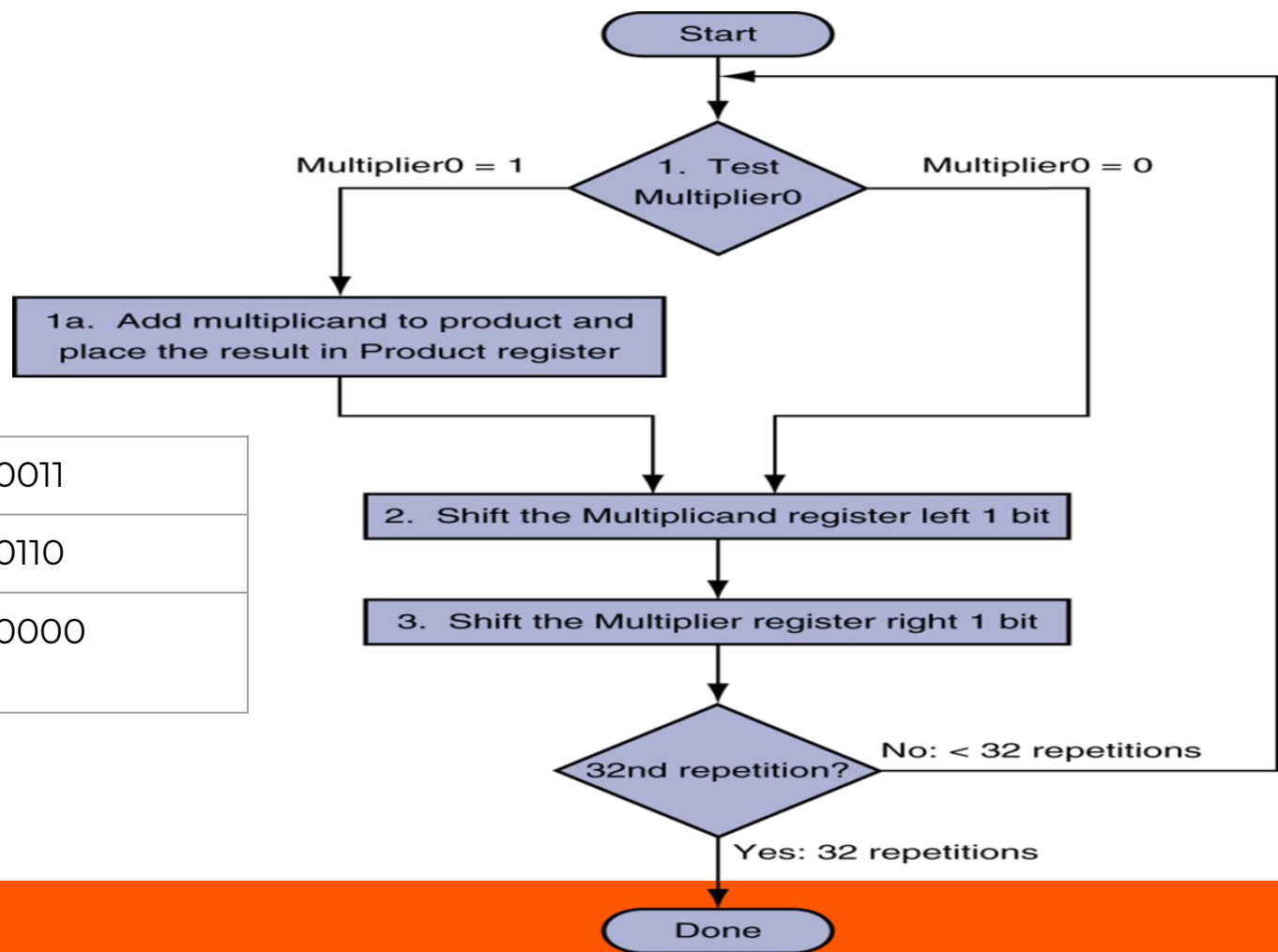
Shift and Add Multiplication Algorithm

Multiplicand	3 (0000 0011)
Multiplier	6 (0000 0110)
Expected Product	18 (0001 0010)



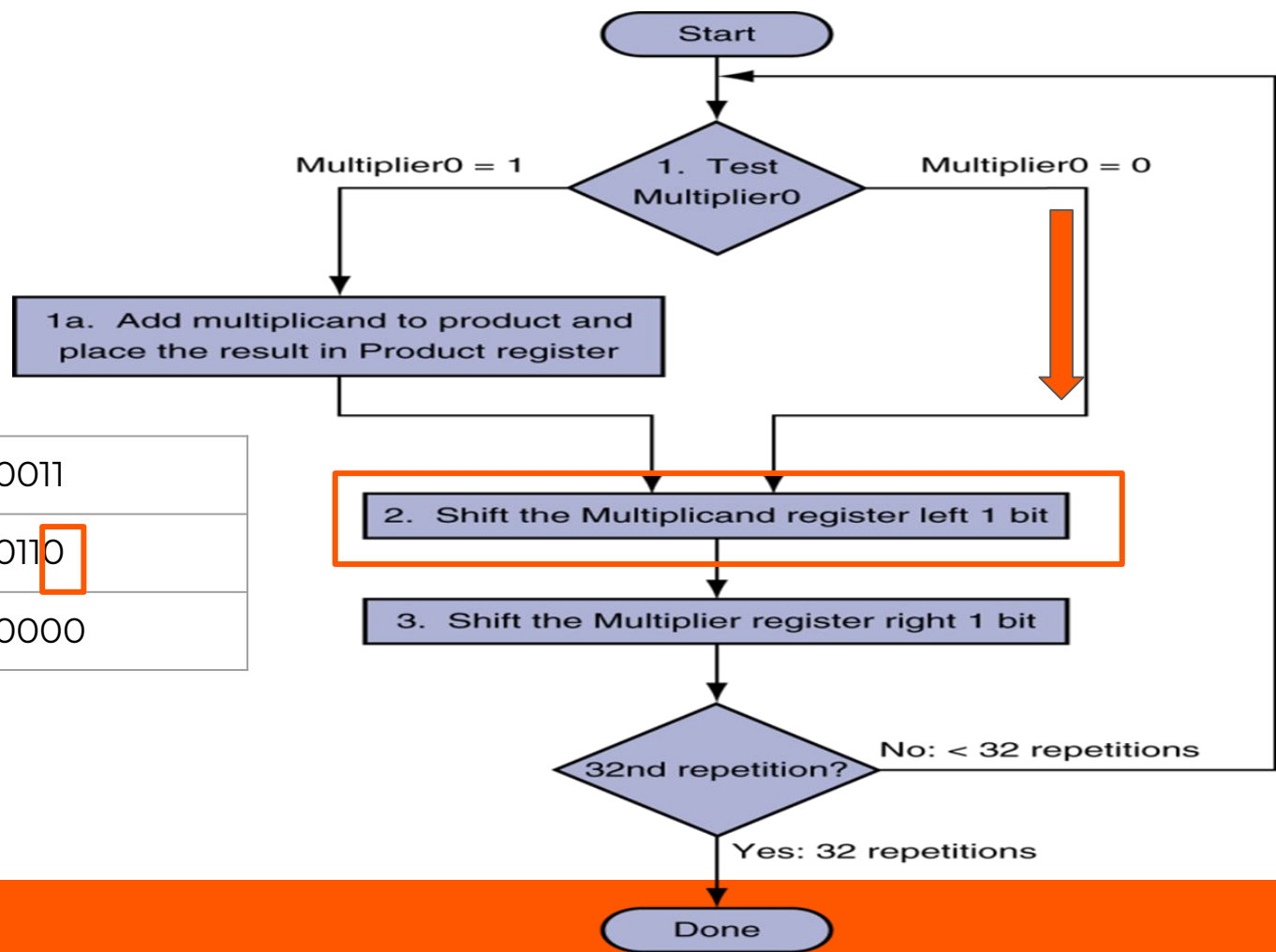
Shift and Add Multiplication Algorithm

Multiplicand	0000 0011
Multiplier	0000 0110
Initial Product	0000 0000



Shift and Add Multiplication Algorithm

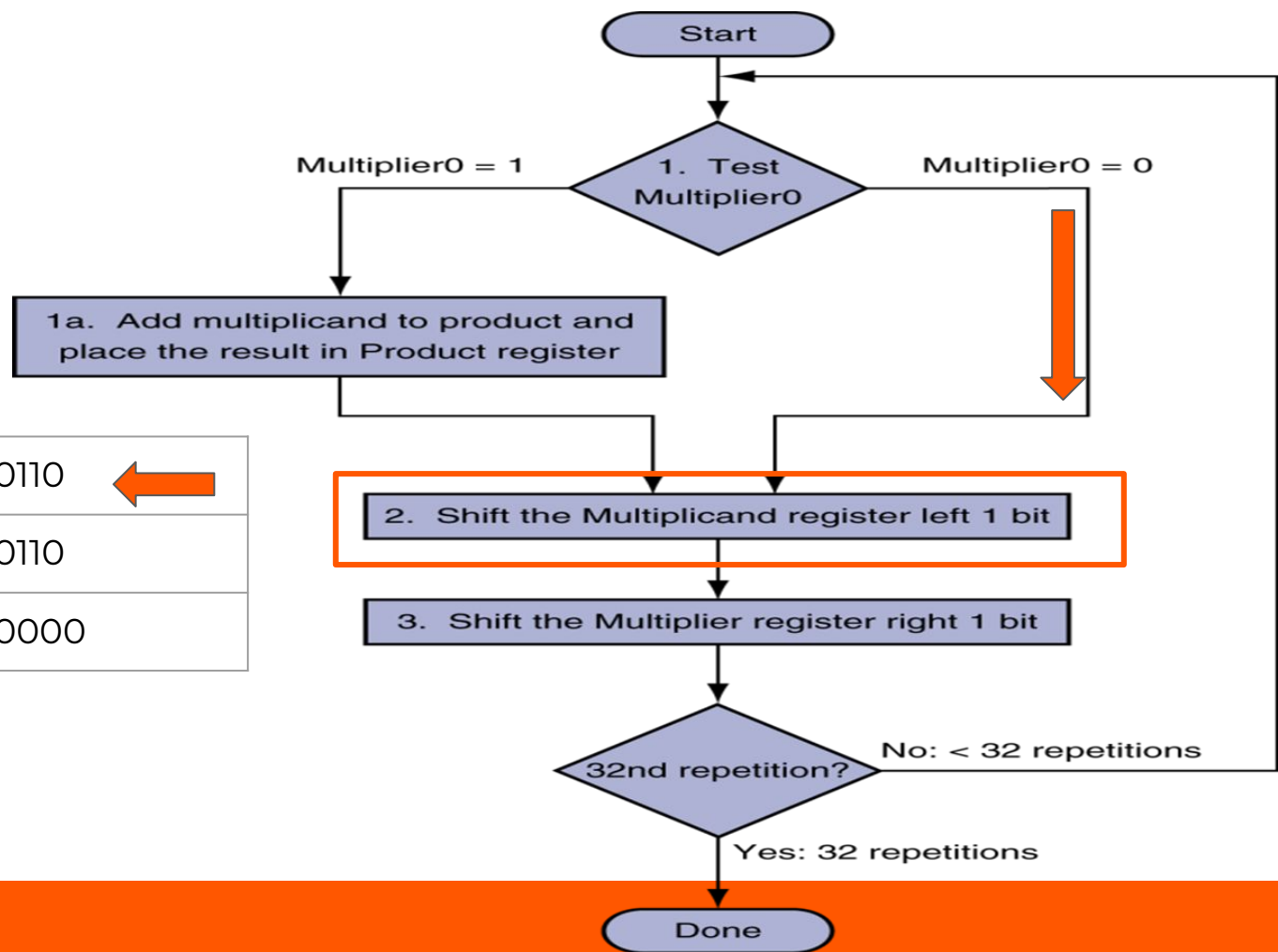
Multiplicand	0000 0011
Multiplier	0000 0110
Initial Product	0000 0000



Shift and Add Multiplication Algorithm

Shift the Multiplicand register left 1 bit

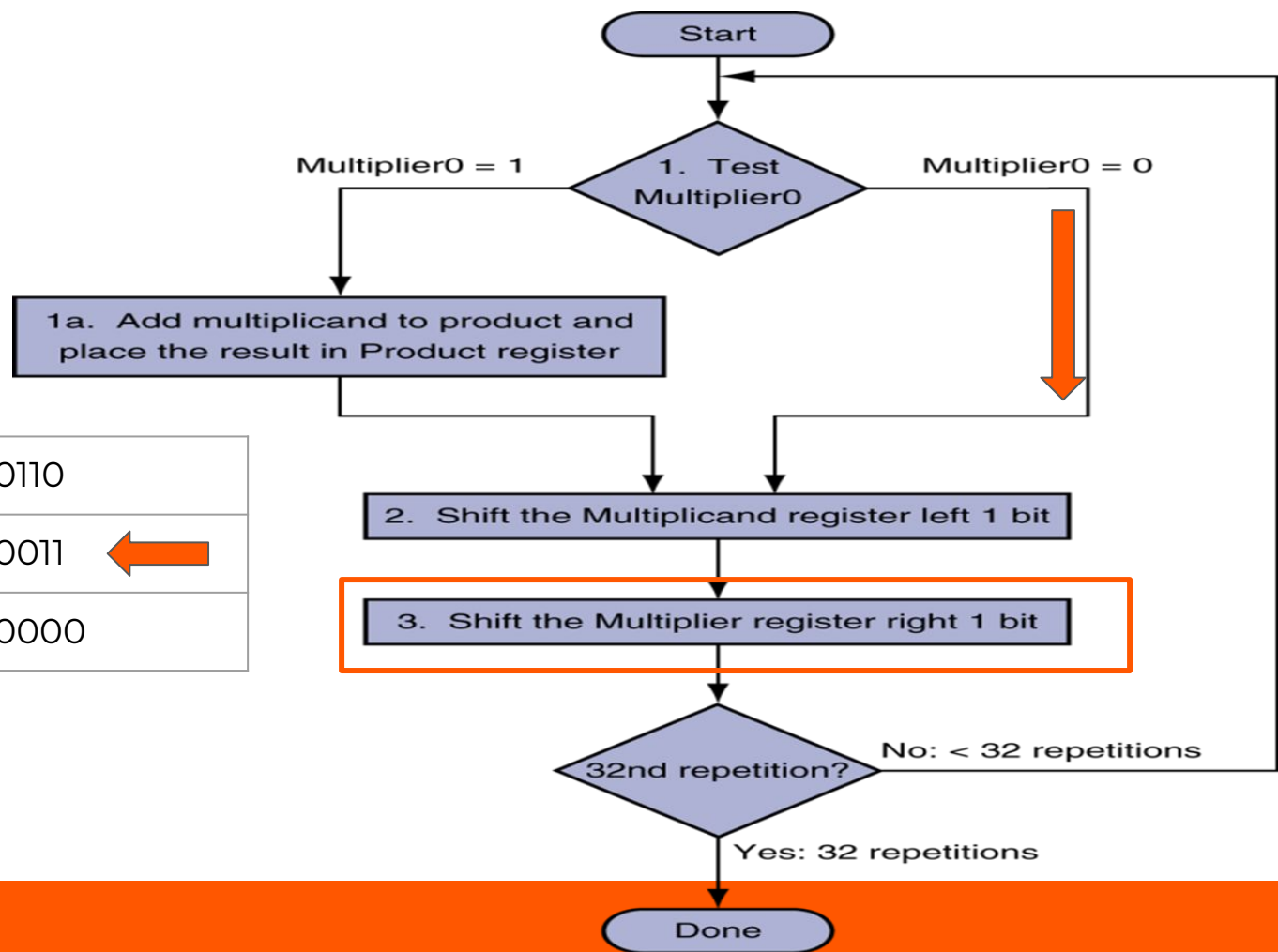
Multiplicand	0000 0110	←
Multiplier	0000 0110	
Initial Product	0000 0000	



Shift and Add Multiplication Algorithm

Shift the Multiplier register right 1 bit

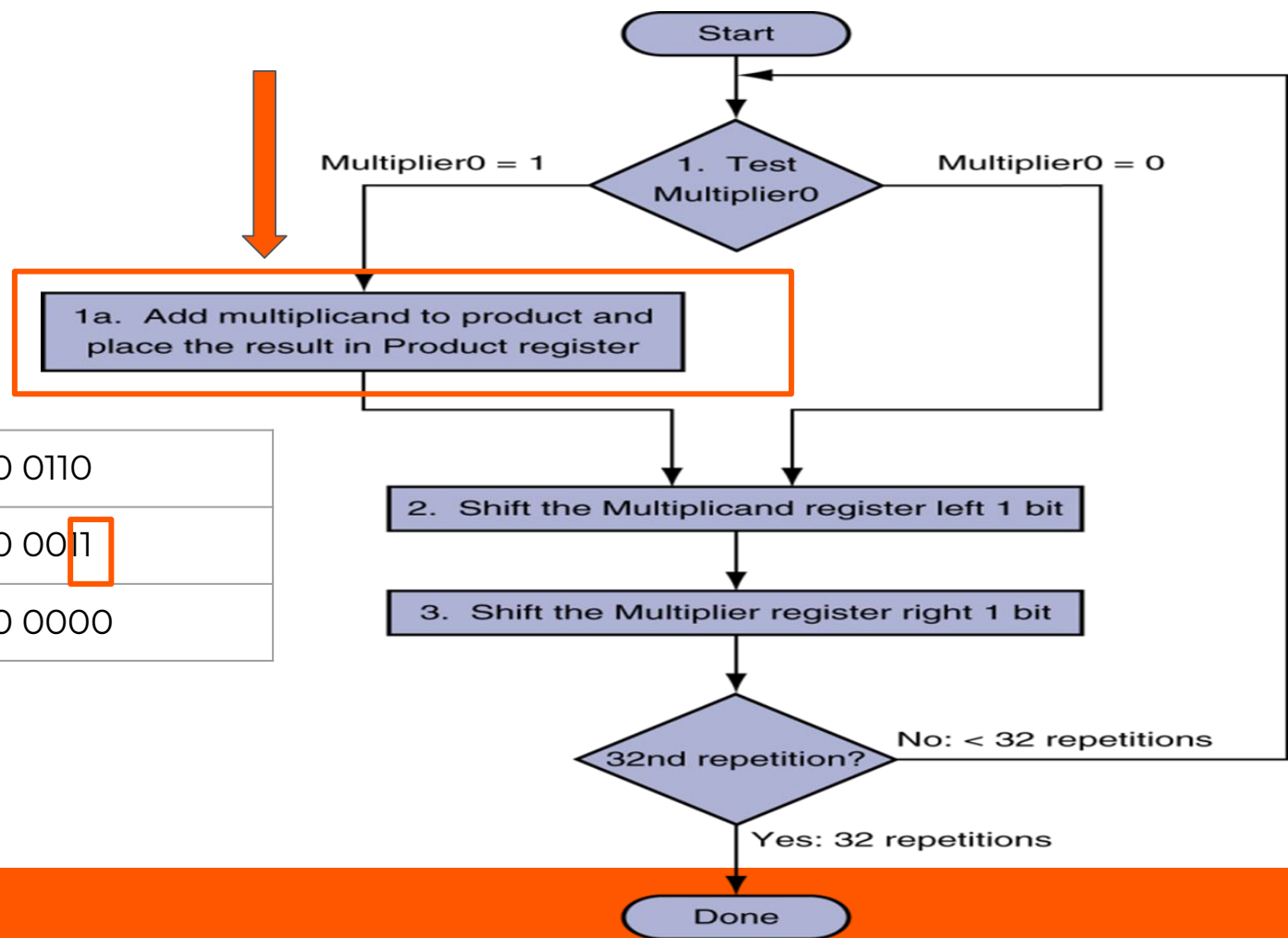
Multiplicand	0000 0110
Multiplier	0000 0011 ←
Initial Product	0000 0000



Shift and Add Multiplication Algorithm

Add multiplicand
To product

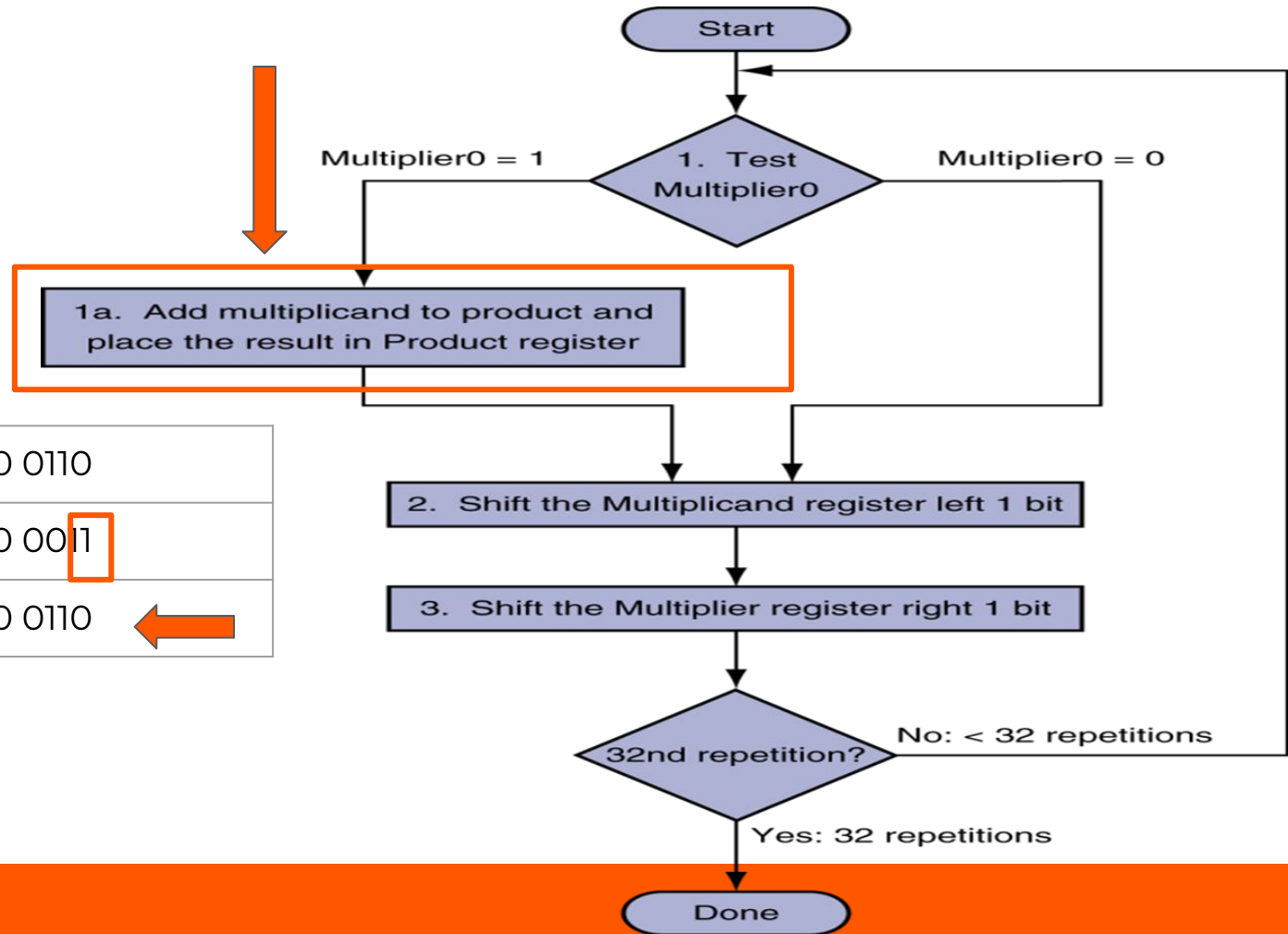
Multiplicand	0000 0110
Multiplier	0000 0011
Initial Product	0000 0000



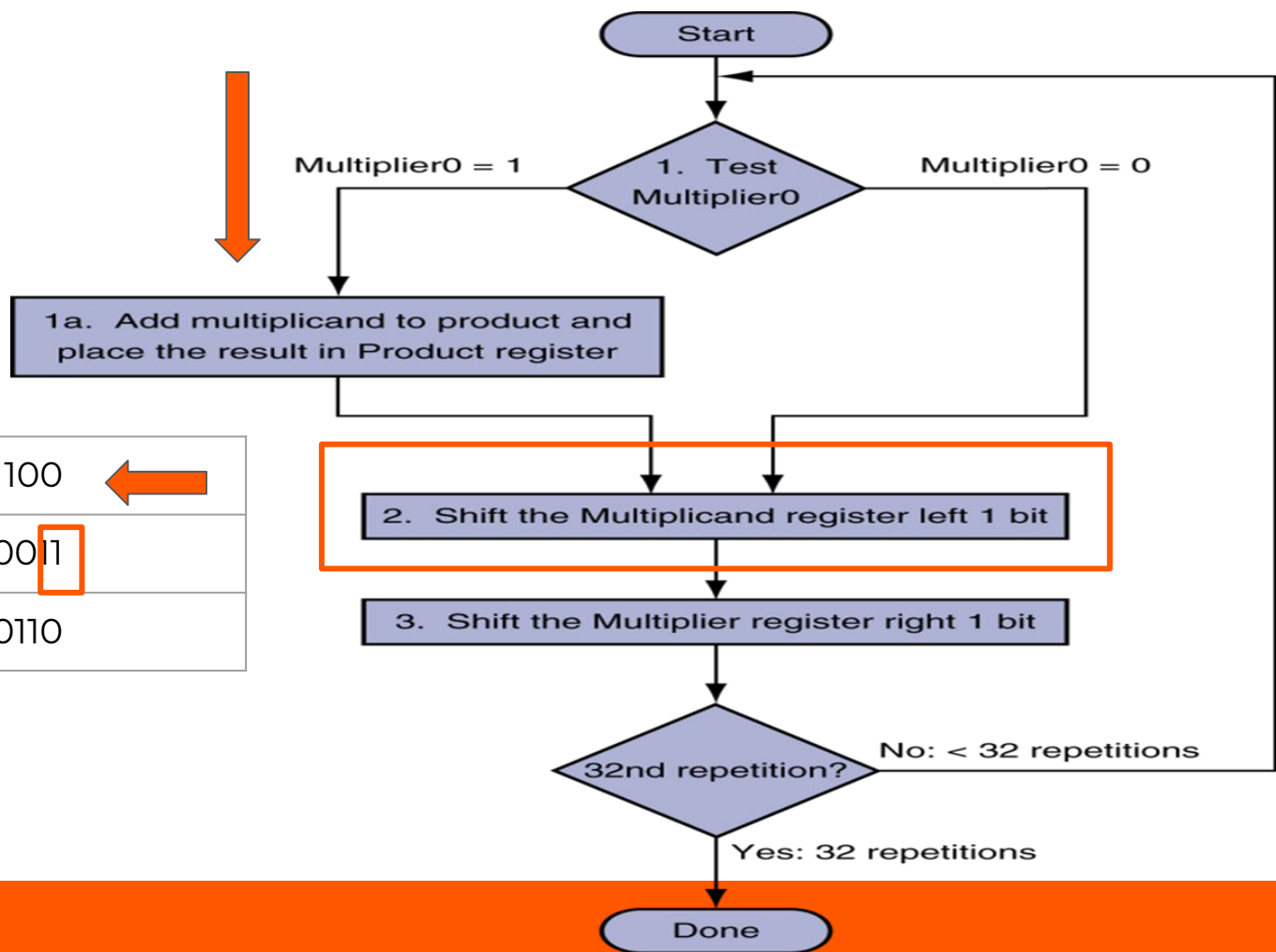
Shift and Add Multiplication Algorithm

Add multiplicand
To product

Multiplicand	0000 0110
Multiplier	0000 0011
Initial Product	0000 0110



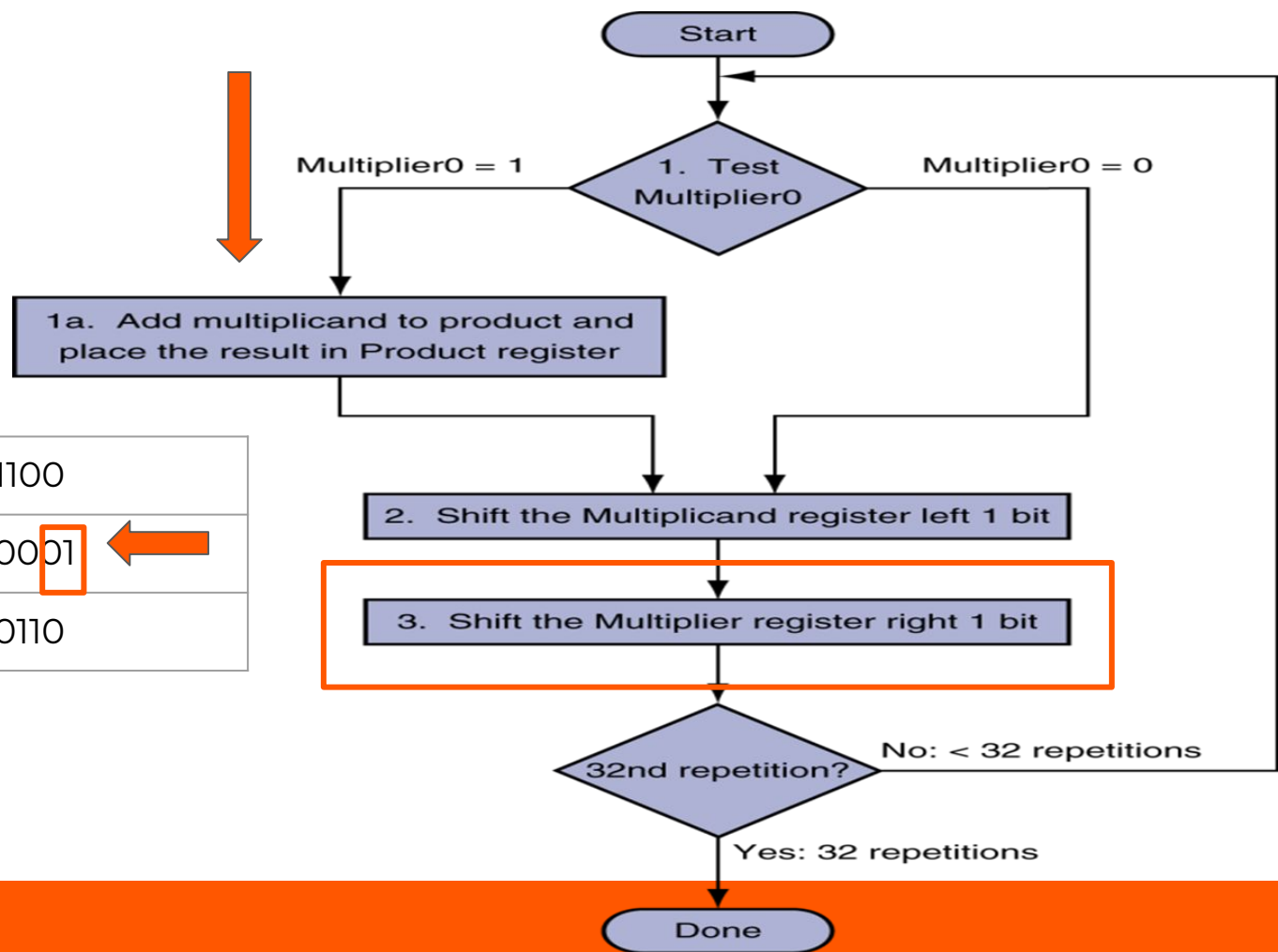
Shift and Add Multiplication Algorithm



Multiplicand	0000 1100	←
Multiplier	0000 0011	
Initial Product	0000 0110	

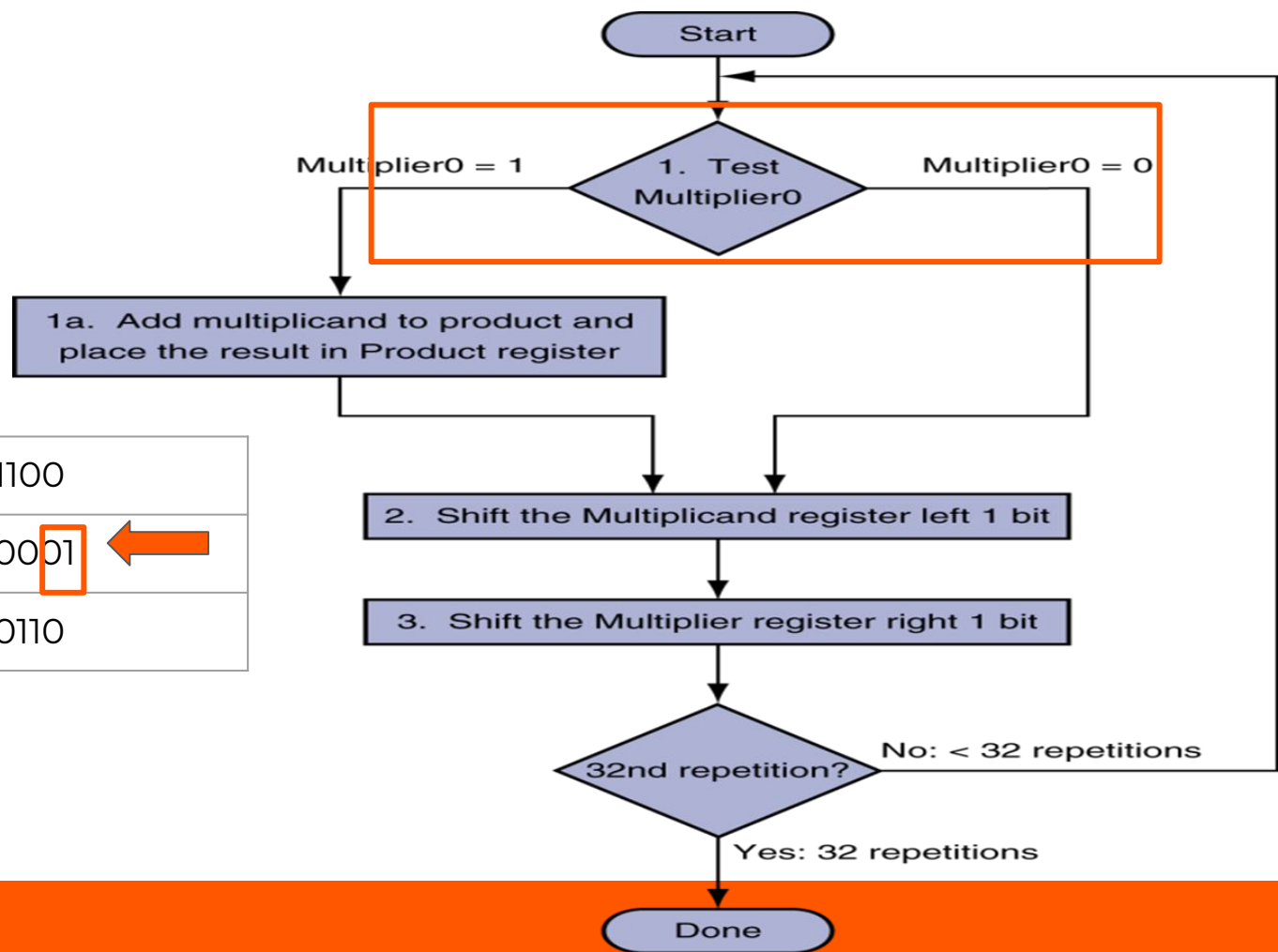
Shift and Add Multiplication Algorithm

Multiplicand	0000 1100
Multiplier	0000 0001 ←
Initial Product	0000 0110

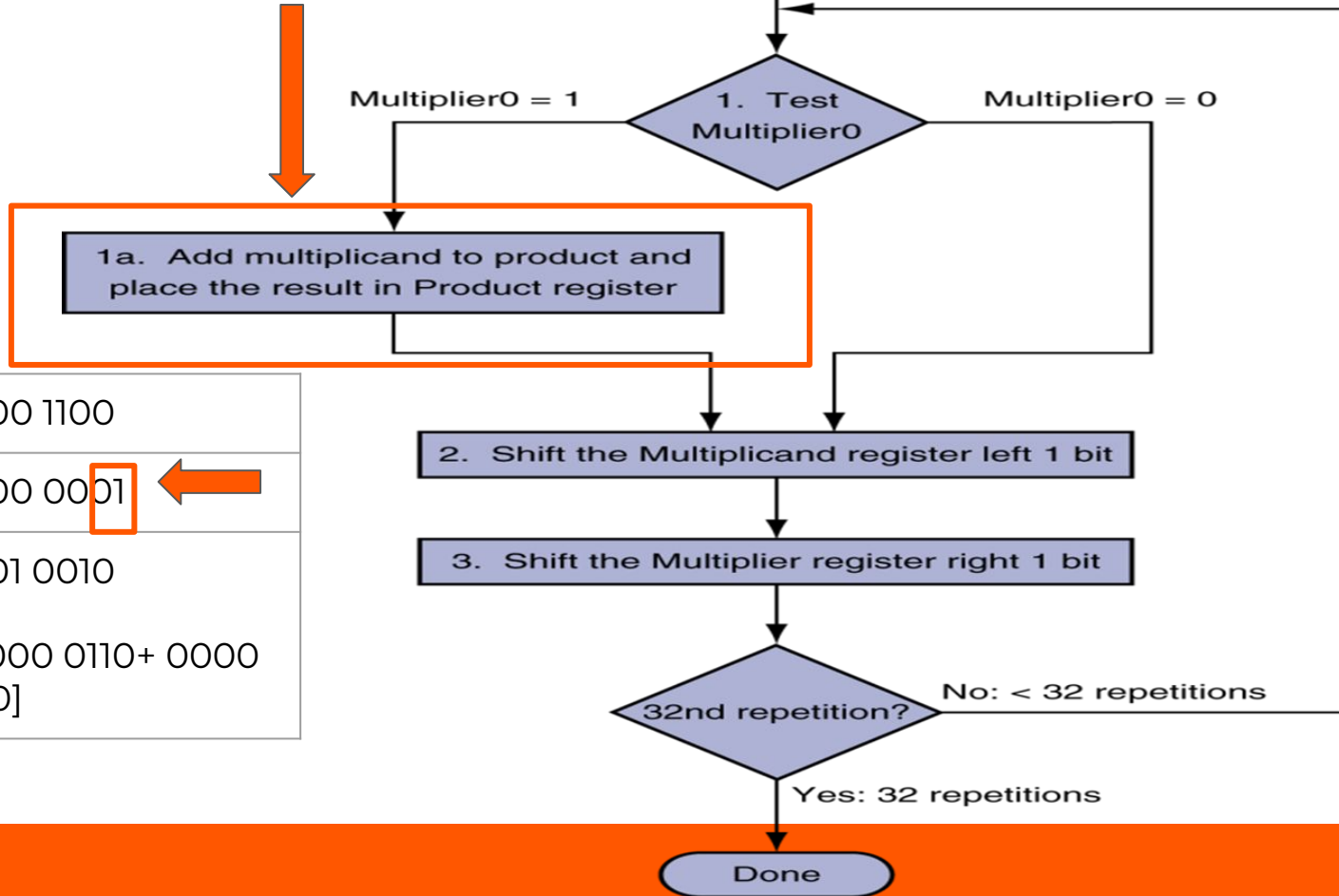


Shift and Add Multiplication Algorithm

Multiplicand	0000 1100
Multiplier	0000 0001 ←
Initial Product	0000 0110

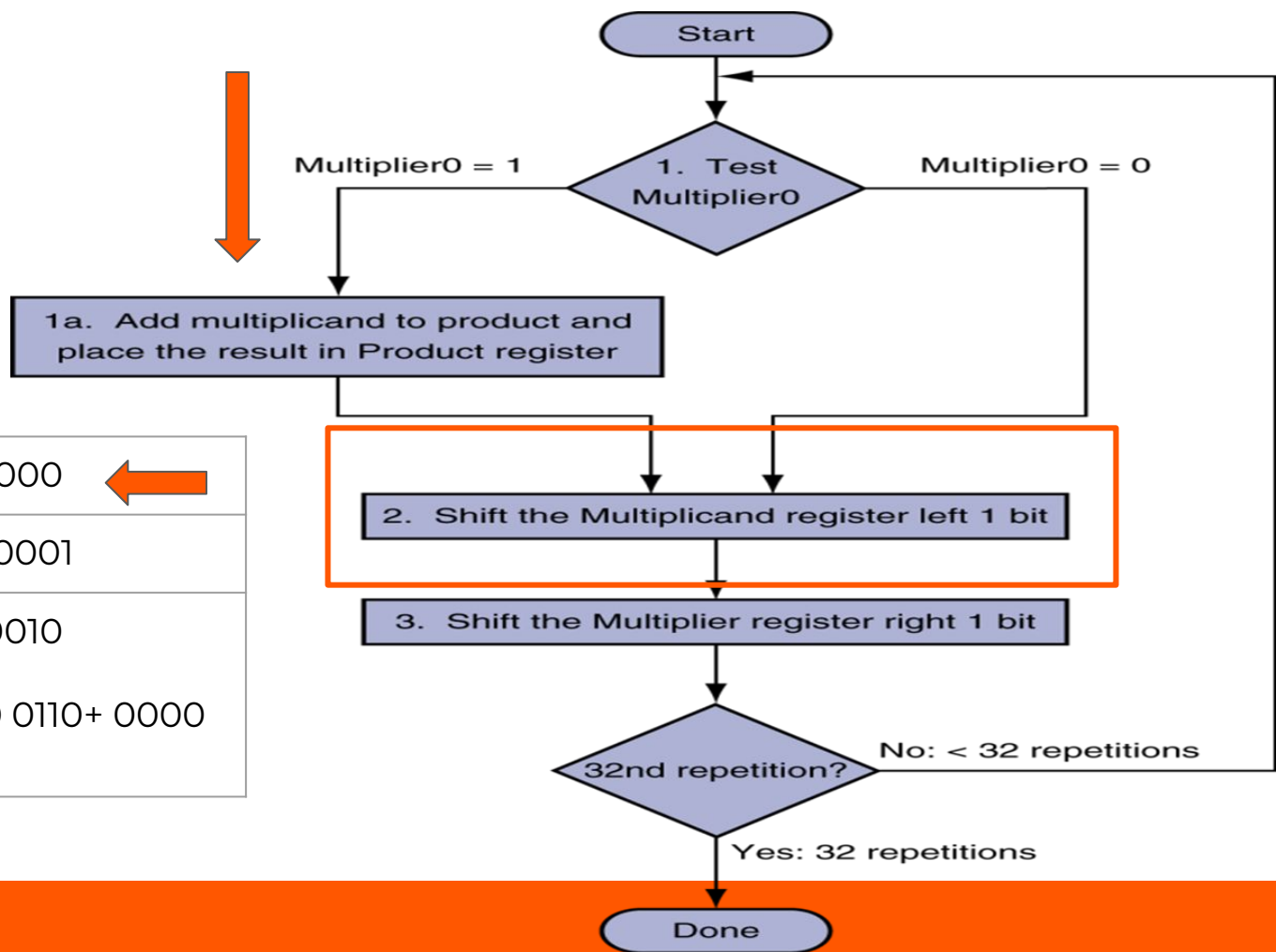


Shift and Add Multiplication Algorithm



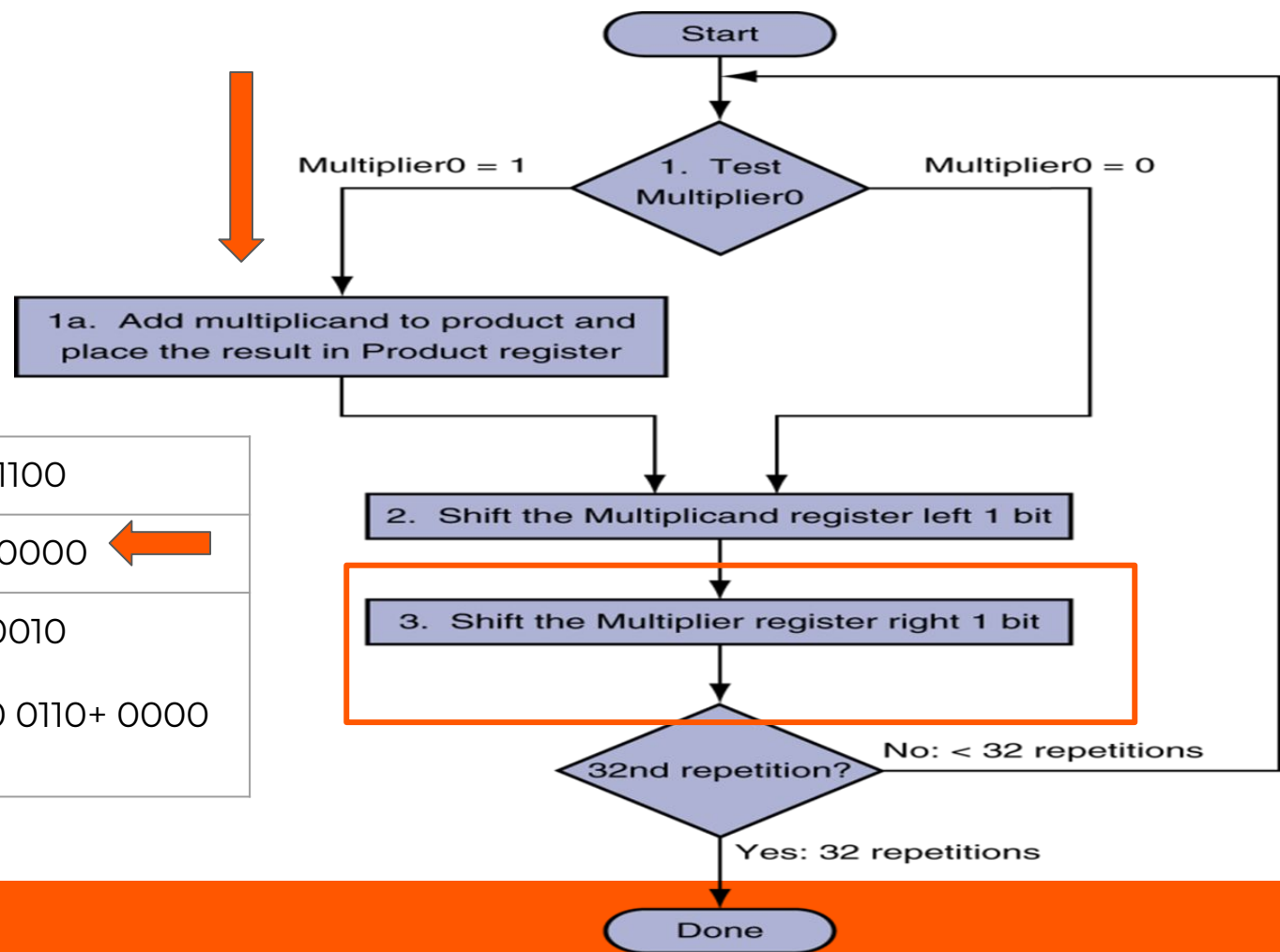
Multiplicand	0000 1100
Multiplier	0000 0001 ←
Initial Product	0001 0010 [0000 0110+ 0000 1100]

Shift and Add Multiplication Algorithm



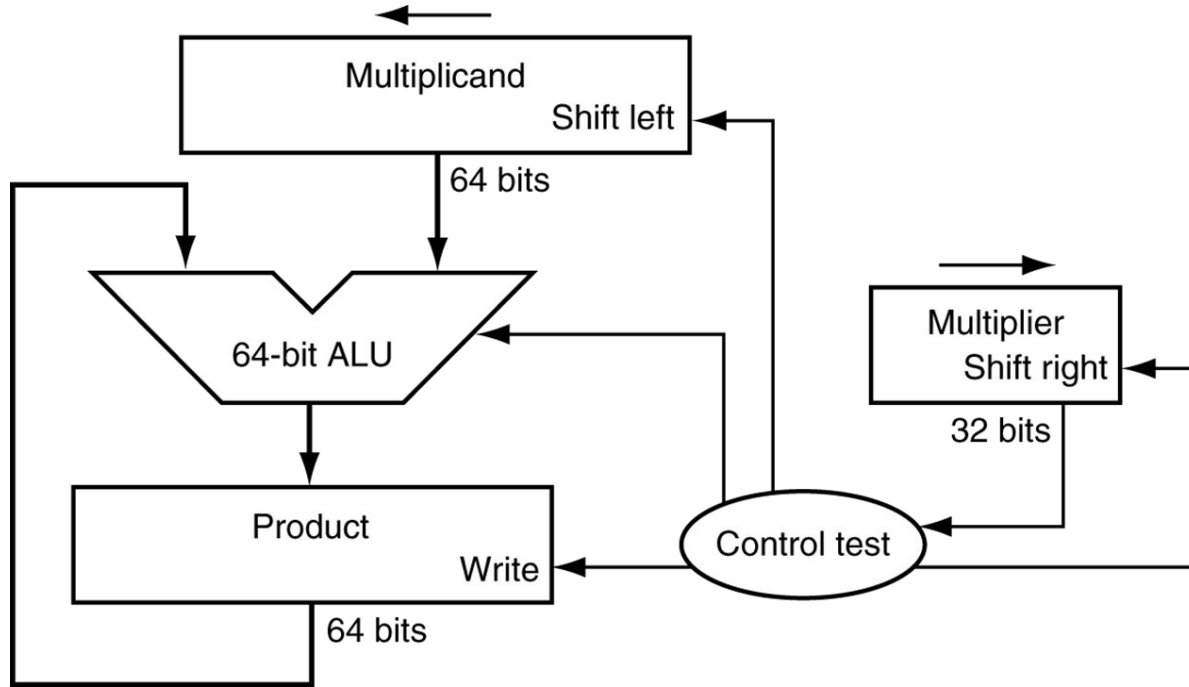
Multiplicand	0001 1000 ←
Multiplier	0000 0001
Initial Product	0001 0010 [0000 0110+ 0000 1100]

Shift and Add Multiplication Algorithm

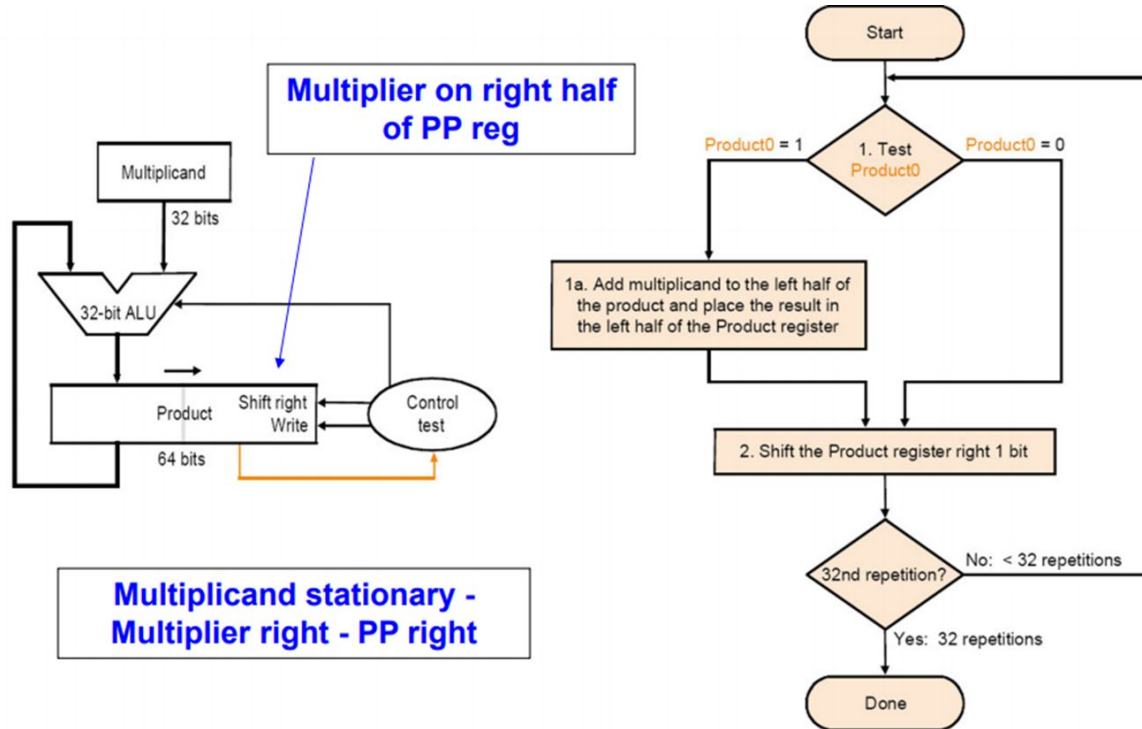


Multiplicand	0000 1100
Multiplier	0000 0000 ←
Initial Product	0001 0010 [0000 0110+ 0000 1100]

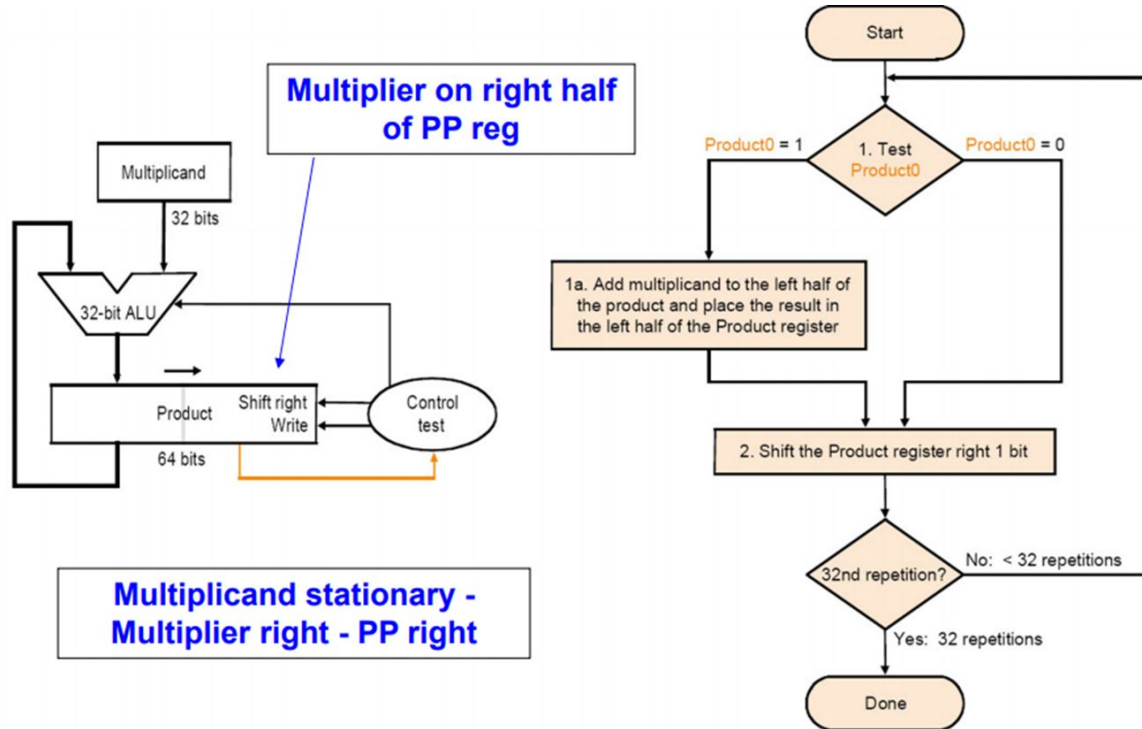
Multiplication Hardware



Optimized Multiplier



Optimized Multiplier



Iter	A (upper)	Q (lower)	Q ₀	Action	Before shift [A Q]	After shift [A Q]
1	0000	0110	0	No add	0000 0110	0000 0011
2	0000	0011	1	A = A + M = 0000+0011 = 0011	0011 0011	0001 1001
3	0001	1001	1	A = A + M = 0001+0011 = 0100	0100 1001	0010 0100
4	0010	0100	0	No add	0010 0100	0001 0010

How is optimization ensured?

Instead of three registers, we:

Use a single 64-bit Product Register:

- Left half accumulates the result.
- Right half holds the multiplier at the start.

The multiplicand is stationary (never shifted).

The Product Register is shifted right after each cycle.

Multiplier bits are read from the LSB (least significant bit) of the Product Register.

Fewer components = simpler and more efficient.

mult / multu instructions

- mult rs, rt → multiply signed 32-bit integers in registers rs and rt.
- multu rs, rt → multiply unsigned 32-bit integers.

Two registers for the product (HI and LO)

When you multiply two 32-bit numbers, the result can be up to 64 bits.

MIPS uses two special registers:

- HI → upper 32 bits of the product
- LO → lower 32 bits of the product

So the product isn't stored in a normal register immediately, but split between HI and LO.

mult / multu instructions

- mult rs, rt → multiply signed 32-bit integers in registers rs and rt.
- multu rs, rt → multiply unsigned 32-bit integers.

Getting the result back:

mfhi rd → move contents of HI into general register rd.

mflo rd → move contents of LO into general register rd.

Usually, for normal 32-bit multiplication, we care about the LO part (the lower 32 bits).

We can write pseudo instruction

`mul rd, rs, rt`

Perform multiplication (mult rs, rt).

Copy only the LO part into rd.



Thank you!